SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Krishna Tejesh Reddy . Y	**Reg. No.:**	192411200
Date:	19-8-2026	**Duration:**	60 minutes

Code of Conduct
I __ Krishna Tejesh Reddy (192411200)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:  _____

Annexure A – Code Review & Repository Evaluation

CAMPUS PLACEMENT AND RECRUITMENT PORTAL

1. PROBLEM OVERVIEW

1.1 Problem Statement

The Campus Placement and Recruitment Portal is a web-based application developed to simplify and manage the campus recruitment process in an organized manner. Traditional placement processes often depend on manual registration, spreadsheets, emails, physical documents, and separate communication channels. This can make it difficult to maintain accurate student records, manage company information, publish job opportunities, and track applications.

The proposed Campus Placement and Recruitment Portal provides a centralized platform for students, recruiters, and placement officers. Students can register, create profiles, view available job opportunities, check eligibility, and apply for suitable positions. Recruiters can manage company information, publish job vacancies, define eligibility requirements, and review student applications. Placement officers can manage students, recruiters, job postings, applications, and placement activities.

The system therefore reduces manual effort and provides a structured platform for managing the complete campus recruitment process.

1.2 Implemented Solution

The implemented solution consists of a web application containing separate functionalities for students, recruiters, and placement officers.

The major components include:

- Student registration and login.
- Recruiter registration and login.
- Placement officer management.
- Student profile management.
- Company profile management.
- Job posting and management.
- Job eligibility checking.
- Job application management.
- Application status tracking.
- Placement record management.
- Database management.
- Repository-based version control.

1.3 Project Objectives

The major objectives of the project are:

- To develop a centralized campus recruitment platform.
- To simplify student registration and profile management.
- To provide students with available placement opportunities.
- To allow students to apply for eligible job positions.
- To allow recruiters to publish job vacancies.
- To help recruiters manage student applications.
- To help placement officers monitor recruitment activities.
- To reduce manual paperwork.

- To improve communication between students, recruiters, and placement officers.
- To maintain recruitment information systematically.

1.4 Key Functionalities

Module	Key Functionalities
Student	Registration, login, profile management, job viewing, eligibility checking, job application, application tracking
Recruiter	Registration, company profile, job posting, eligibility criteria, application viewing, candidate shortlisting
Placement Officer	Student management, recruiter management, job management, application monitoring, placement management
Database	Stores student, recruiter, company, job, application, and placement information
Authentication	Provides secure access to different user roles

1.5 Expected Outcome

The expected outcome of the project is a functional and maintainable recruitment portal that allows the different stakeholders to manage placement activities through a centralized system.

The system should:

- Provide easy access to job opportunities.
- Reduce manual placement activities.
- Maintain accurate recruitment information.
- Improve application tracking.
- Provide better communication between stakeholders.
- Support organized placement management.

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

The project repository is organized into separate directories for frontend, backend, database, configuration, and documentation.

```
campus-placement-portal/
│
├── frontend/
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── student-dashboard.html
│   ├── recruiter-dashboard.html
│   ├── admin-dashboard.html
│   ├── jobs.html
│   ├── profile.html
│   └── css/
│       └── style.css
│
├── backend/
│   ├── server.js
│   ├── routes/
│   │   ├── auth.js
│   │   ├── students.js
│   │   ├── recruiters.js
│   │   └── jobs.js
│   │
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── studentController.js
│   │   ├── recruiterController.js
│   │   └── jobController.js
│   │
│   └── config/
│       └── database.js
│
├── database/
│   └── schema.sql
│
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── package.json
└── README.md
```

2.2 Repository Organization Evaluation

The repository is organized according to the different responsibilities of the application.

Folder/File	Purpose	Maintainability
frontend/	Contains user interface files.	High
backend/	Contains server-side logic.	High
routes/	Organizes API endpoints.	High
controllers/	Separates application logic.	High
config/	Contains configuration files.	High
database/	Contains database-related files.	High
Dockerfile	Defines container configuration.	High
docker-compose.yml	Manages application services.	High
.gitignore	Prevents unnecessary files from being tracked.	High
README.md	Provides project documentation.	High

2.3 File Naming Conventions

The project follows descriptive file names that identify the responsibility of each file.

Examples include:

- studentController.js
- recruiterController.js
- jobController.js
- database.js
- student-dashboard.html
- recruiter-dashboard.html

Descriptive naming improves:

- Code readability.
- Navigation through the repository.
- Collaboration.
- Maintenance.
- Debugging.

2.4 Maintainability and Collaboration

The repository structure supports maintainability because:

- Related files are grouped together.
- Frontend and backend responsibilities are separated.
- Database files are maintained independently.
- Configuration files are separated from application logic.
- Developers can work on individual modules.
- New features can be added without significantly affecting existing modules.

3. CODE QUALITY REVIEW

3.1 Code Readability

Code readability is important for understanding, modifying, and maintaining the application.

The project follows readable coding practices through:

- Meaningful variable names.
- Descriptive function names.
- Logical file organization.
- Consistent indentation.
- Separation of frontend and backend code.
- Modular organization of backend functionality.

3.2 Code Modularity

The application is divided into modules based on functionality.

The major modules include:

- Authentication module.
- Student module.
- Recruiter module.
- Job module.
- Application module.

- Database module.

This modular structure makes it easier to:

- Add new features.
- Modify existing functionality.
- Identify errors.
- Test individual components.
- Reuse application logic.

3.3 Coding Standards

The following coding practices are considered during the review:

- Consistent indentation.
- Meaningful naming conventions.
- Proper file organization.
- Avoidance of unnecessary duplicate code.
- Clear function structure.
- Secure handling of configuration information.

3.4 Code Quality Evaluation

Criteria	Observation	Evaluation
Readability	Code is organized using descriptive names and structured files.	Good
Modularity	Application functionality is separated into modules.	Good
Naming	Files and functions use meaningful names.	Good
Organization	Frontend, backend, and database are separated.	Good
Maintainability	Modular structure makes future modifications easier.	Good
Documentation	README and setup information provide project guidance.	Good
Error Handling	Errors should be handled appropriately at application and database levels.	Can be improved
Security	Authentication and configuration practices should be continuously reviewed.	Can be improved

3.5 Strengths

The major strengths identified during the code review are:

- Clear separation of application components.
- Modular backend organization.
- Descriptive file naming.
- Organized repository structure.
- Separate database configuration.
- Docker support.
- Git-based version control.
- Easy addition of new modules.

3.6 Areas for Improvement

The following improvements can further enhance code quality:

- Increase automated test coverage.
- Improve error handling.
- Use environment variables for sensitive configuration.
- Reduce duplicate code.
- Add validation to all user inputs.
- Improve documentation for complex functions.
- Apply consistent coding standards throughout the project.
- Introduce automated code quality checking.

4. VERSION CONTROL EVALUATION

4.1 Git Repository Evaluation

Git is used to maintain the development history of the Campus Placement and Recruitment Portal.

The repository should contain:

- Complete project source code.
- Meaningful commit history.

- Feature branches.
- Development branch.
- Main branch.
- Project documentation.
- Docker configuration.

4.2 Branching Strategy

The recommended branching structure is:

```
main
 │
 └── development
     │
     ├── feature/student-module
     ├── feature/recruiter-module
     ├── feature/job-module
     └── feature/application-module
```

Branch	Purpose
main	Stable project version
development	Integration and testing
feature/student-module	Student functionality
feature/recruiter-module	Recruiter functionality
feature/job-module	Job management
feature/application-module	Application functionality

4.3 Commit History

A meaningful commit history can contain:

Initial project setup

Add database schema

Add student registration module

Add student login functionality

Add recruiter module

Add job posting functionality

Add job application functionality

Add placement officer dashboard

Add Docker configuration

Fix database connection

Improve authentication validation

Prepare project release

4.4 Commit Message Evaluation

Good commit messages should:

- Clearly describe the change.
- Be concise.
- Focus on one logical change.
- Avoid vague descriptions.
- Help developers understand project history.

Example:

```
git commit -m "Add student registration module"
```

A meaningful commit message is more useful than:

```
git commit -m "updated files"
```

4.5 Version Control Benefits

Git supports collaborative development by:

- Allowing multiple developers to work simultaneously.
- Providing separate feature branches.
- Tracking all modifications.
- Supporting merging.
- Allowing previous versions to be restored.
- Providing a record of project development.
- Supporting code review and collaboration.

5. CODE TESTING AND VALIDATION

5.1 Testing Objective

Testing is performed to verify that the Campus Placement and Recruitment Portal functions according to the expected requirements.

Testing focuses on:

- User authentication.
- Student registration.
- Recruiter registration.
- Profile management.
- Job posting.
- Job searching.
- Job applications.
- Application tracking.
- Database operations.

5.2 Test Cases

Test ID	Test Case	Input/Action	Expected Result
TC01	Student Registration	Enter valid student details	Student account created
TC02	Student Login	Enter valid credentials	Student dashboard displayed
TC03	Invalid Login	Enter incorrect credentials	Error message displayed
TC04	Student Profile	Update profile details	Profile updated successfully
TC05	Job Listing	Open available jobs	Job listings displayed
TC06	Eligibility	Check student eligibility	Eligibility result displayed
TC07	Job Application	Apply for eligible job	Application submitted
TC08	Application Status	Open application history	Current status displayed
TC09	Recruiter Login	Enter recruiter credentials	Recruiter dashboard displayed
TC10	Job Posting	Enter job details	New vacancy created
TC11	Application Review	Recruiter views applications	Applications displayed
TC12	Database	Store and retrieve records	Correct data returned

5.3 Testing Results

Test Category	Result
Registration	Successful
Login	Successful
Profile Management	Successful
Job Listing	Successful
Eligibility Checking	Successful
Job Application	Successful
Recruiter Management	Successful
Job Posting	Successful
Application Tracking	Successful
Database Operations	Successful

5.4 Validation

The application is considered functional when:

- Users can register successfully.
- Users can log in using valid credentials.
- Incorrect credentials are rejected.
- Job opportunities are displayed correctly.
- Eligible students can apply for jobs.
- Recruiters can create job postings.
- Applications are stored correctly.
- Application information can be retrieved.
- Different modules communicate correctly.

5.5 Testing Evidence

The following screenshots can be included in the report:

- Registration page.
- Login page.
- Student dashboard.
- Recruiter dashboard.
- Job listing page.

- Job application page.
- Application status page.
- Database records.
- Git repository.
- Git commit history.
- Docker containers.

The assignment specifically expects test execution screenshots and evidence of application functionality.

6. REPOSITORY DOCUMENTATION

6.1 README Documentation

The README file should provide sufficient information for another developer to understand and run the project.

The README should contain:

- Project title.
- Project description.
- Objectives.
- Technologies used.
- System requirements.
- Installation procedure.
- Database configuration.
- Application execution steps.
- Git commands.
- Docker commands.
- Testing procedure.
- Project structure.

6.2 Installation Instructions

A typical installation process consists of:

- Clone the repository.
- Open the project directory.
- Install required dependencies.
- Configure the database.
- Configure environment variables.
- Start the backend.
- Start the frontend.
- Verify the application.

Example:

```
git clone <repository-url>
cd campus-placement-portal
npm install
```

The application can then be started using the appropriate project command.

6.3 Docker Installation

The project can also be executed using Docker.

```
docker build -t campus-placement-portal .
docker compose up -d
```

Running containers can be checked using:

```
docker ps
```

6.4 Documentation Evaluation

Documentation Area	Evaluation
Project Description	Clearly explains the purpose of the system.
Installation Guide	Provides steps required to run the project.
Usage Guide	Explains major application functions.
Dependencies	Lists required software and packages.
Repository Structure	Explains major folders and files.
Git Instructions	Provides version-control information.
Docker Instructions	Explains container deployment.
Testing	Provides testing information.

6.5 Documentation Improvements

The documentation can be improved by:

- Adding screenshots to the README.
- Providing detailed installation steps.
- Including system requirements.
- Adding API documentation.
- Adding database schema information.
- Including troubleshooting instructions.
- Providing Docker deployment instructions.
- Adding contribution guidelines.
- Including version information.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Overall Code Review Findings

The overall code review indicates that the Campus Placement and Recruitment Portal follows a modular and organized structure.

The major positive findings are:

- The repository is logically organized.
- Application components are separated.
- Modules are easy to identify.
- Git can be used effectively for collaborative development.
- Docker provides a consistent deployment environment.
- The project can be extended with additional modules.

7.2 Code Review Summary

Review Area	Finding	Status
Repository Structure	Logical separation of project components	Good
File Naming	Descriptive and meaningful names	Good
Code Modularity	Functional components are separated	Good
Readability	Structured and understandable organization	Good
Git Usage	Supports version management	Good
Branching	Feature-based development supported	Good
Commit Messages	Should remain descriptive and consistent	Good
Testing	Functional test cases can validate major modules	Good
Documentation	README and setup information required	Good
Security	Authentication and input validation should be strengthened	Improvement Required
Error Handling	More comprehensive error handling can be added	Improvement Required
Automation	CI/CD and automated testing can be introduced	Future Enhancement

7.3 Recommendations for Code Quality

The following recommendations are proposed:

- Follow consistent coding standards.
- Use meaningful variable and function names.
- Avoid duplicate code.
- Separate business logic from presentation logic.
- Add appropriate error handling.

- Validate all user inputs.
- Use secure authentication mechanisms.
- Keep sensitive configuration outside source code.
- Introduce automated testing.
- Perform regular code reviews.

7.4 Recommendations for Repository Management

- Maintain a clean repository structure.
- Use meaningful branch names.
- Use descriptive commit messages.
- Protect the main branch.
- Review code before merging.
- Keep unnecessary files out of the repository.
- Maintain an updated .gitignore.
- Tag stable releases.

7.5 Recommendations for Documentation

- Maintain a complete README.
- Add installation instructions.
- Add screenshots.
- Document application workflows.
- Document API endpoints.
- Document database configuration.
- Add troubleshooting information.
- Update documentation whenever major features change.

7.6 Recommendations for Testing

- Increase test coverage.
- Add unit testing.
- Add integration testing.
- Add API testing.
- Perform validation testing.

- Automate repetitive tests.
- Include security testing.
- Test application performance.

7.7 Future Enhancements

The Campus Placement and Recruitment Portal can be improved through:

- AI-based candidate-job matching.
- Automated resume screening.
- Online aptitude tests.
- Online coding assessments.
- Automated interview scheduling.
- Email and SMS notifications.
- Placement analytics.
- Cloud deployment.
- CI/CD integration.
- Automated testing pipelines.
- Kubernetes deployment.
- Advanced role-based access control.

8. OVERALL EVALUATION

The Campus Placement and Recruitment Portal demonstrates a structured approach to software development through proper repository organization, modular code development, version control, testing, and documentation.

The project can be evaluated across the following areas:

Evaluation Area	Assessment
Problem Analysis	Clearly identifies the campus recruitment problem and proposed solution.
Repository Organization	Provides a logical structure for collaboration and maintenance.
Code Quality	Uses modular organization and meaningful naming practices.
Version Control	Supports branching, commits, merging, and collaborative development.
Testing	Provides functional test cases for major modules.
Documentation	Provides project setup, usage, and development information.
Maintainability	Modular architecture supports future modifications.
Future Development	Can be extended using AI, cloud, CI/CD, and advanced analytics.

9. CONCLUSION

The Code Review and Repository Evaluation of the Campus Placement and Recruitment Portal demonstrates that a well-organized software project requires more than functional source code. Proper repository organization, coding standards, version control, testing, documentation, and maintainability are essential for successful software development.

The repository structure separates frontend, backend, database, configuration, and documentation components, making the project easier to understand and maintain. Modular development allows individual functionalities such as student management, recruiter management, job posting, and application processing to be developed and maintained independently.

Git provides effective version control and supports collaborative development through branches, commits, merging, and repository management. Testing and validation ensure that the major functionalities of the application operate according to the expected requirements.

The code review also identifies areas where the system can be improved, particularly in automated testing, error handling, security, documentation, and development automation. Future enhancements such as AI-based candidate matching, automated resume screening, CI/CD, cloud deployment, and Kubernetes can further improve the system.

Overall, the Campus Placement and Recruitment Portal provides a suitable foundation for an organized, maintainable, and scalable campus recruitment management system.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations &	Insightful findings, practical recommendations, professional report, clear	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Presentation(10 %)	screenshots, formatting, and references.			

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25